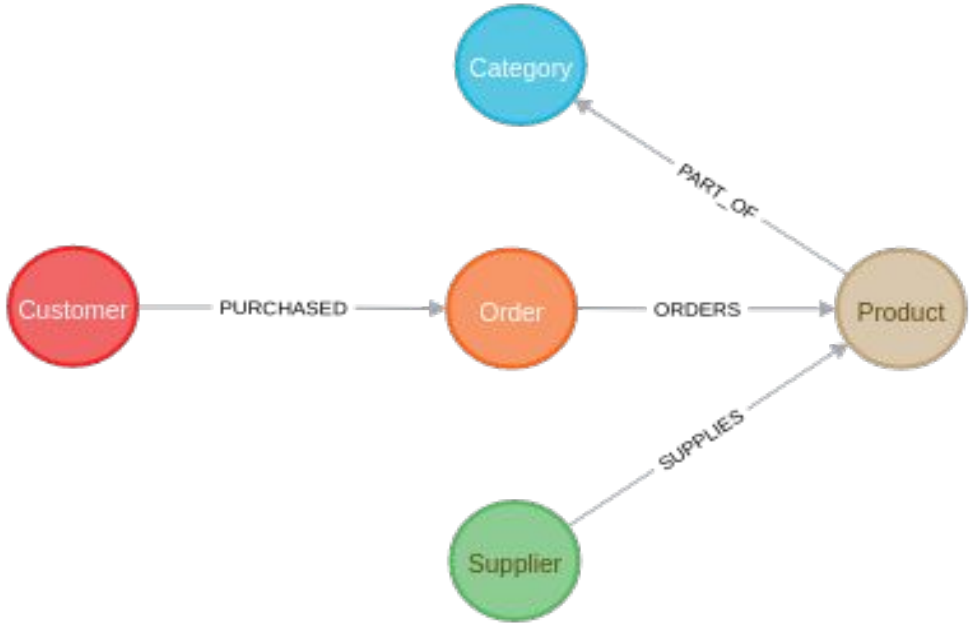
Introduction to Neo4j

Neo4j

Neo4j is a graph database, which is an alternative to a relational (SQL database).

It allows for more complex data relationships compared to a traditional SQL table.

Uses a network of interconnected entities to represent the dataset, stored in a (graph-theory) graph, meaning nodes and edges.

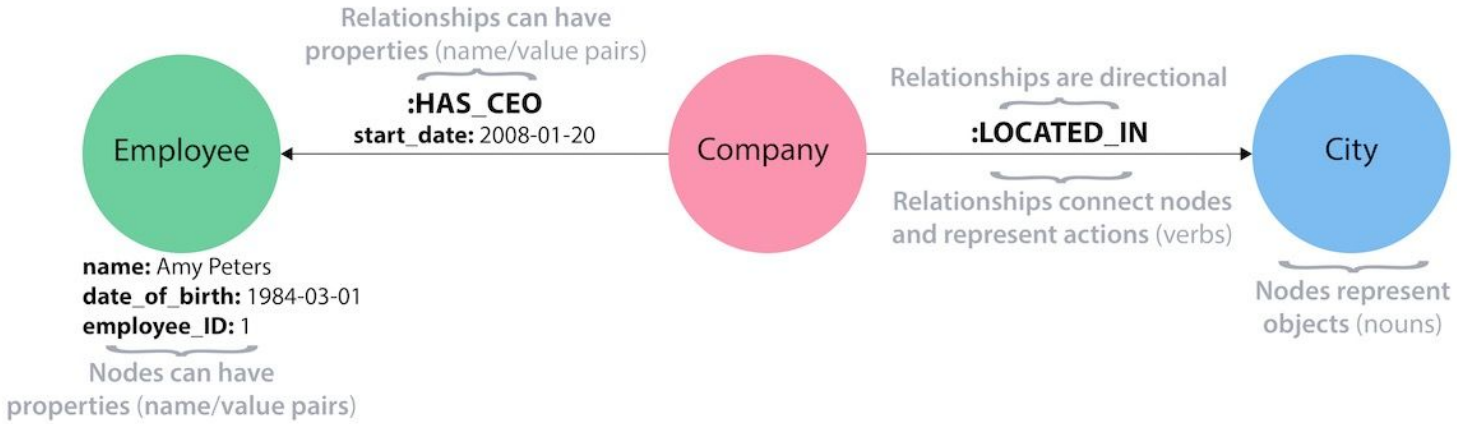


Neo4j

The fundamental building block in a neo4j graph is a **node**.

A node represents some entity (whether it be a customer, supplier, product, etc.).

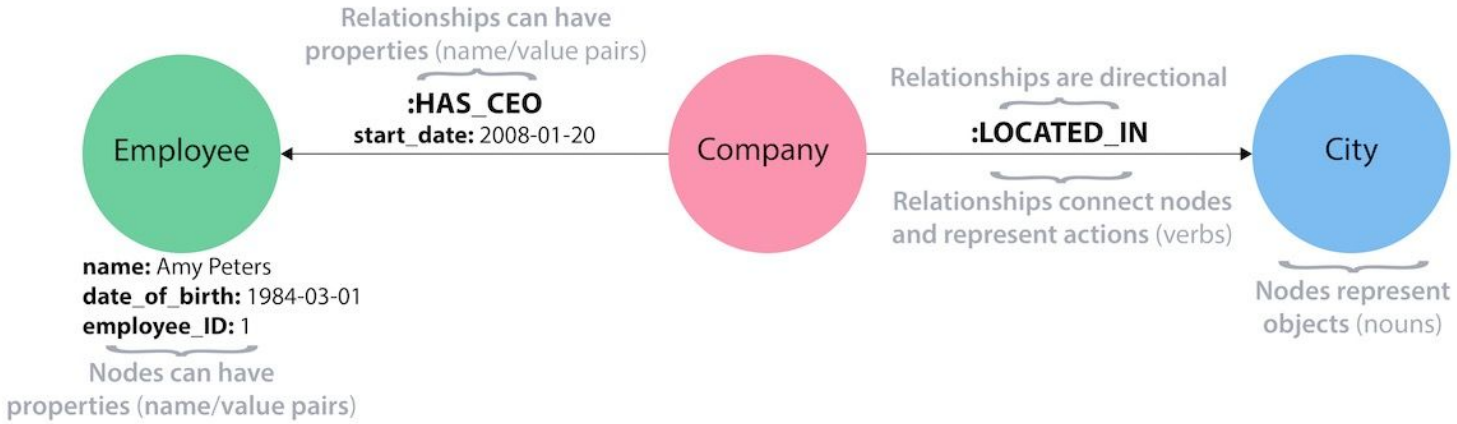
Nodes can have one or more **labels** to identify their type/role (Customer, Order, etc.).



Neo4j

Nodes can also have one or more **properties**, which provide information about those nodes.

Nodes of the same type do *not* have to have the same properties - the schema is flexible.



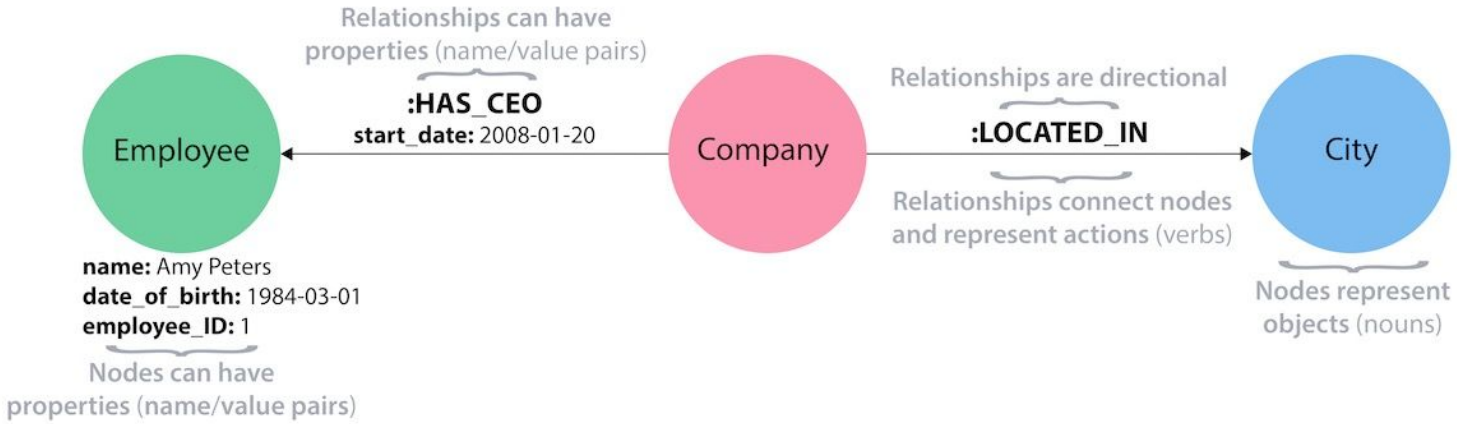
Neo4j

Nodes are linked by directed edges called **relationships**.

Relationships have exactly one relationship type.

Eg.

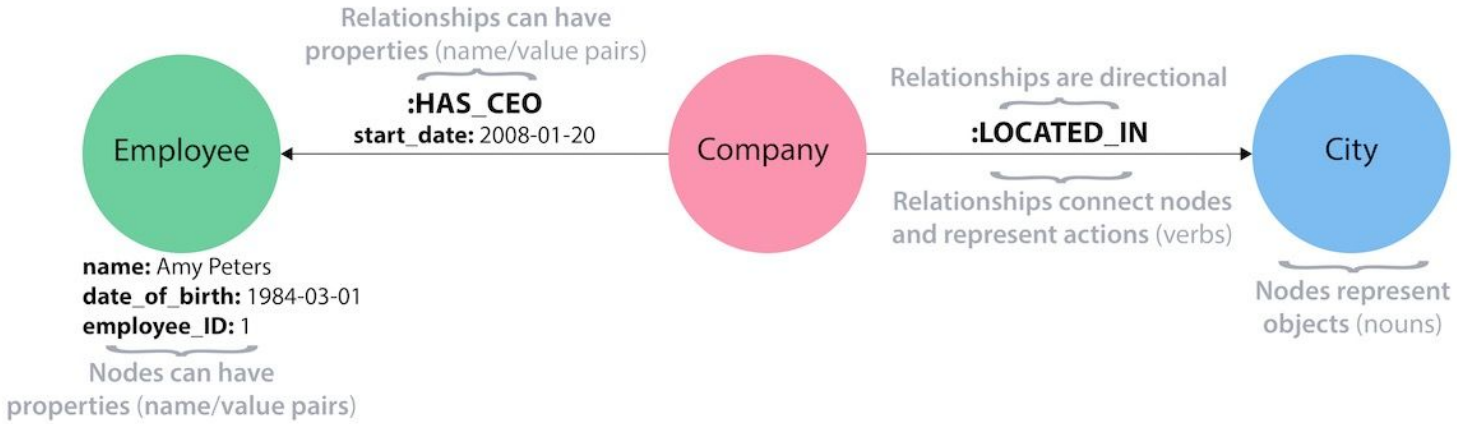
- Customer **Purchased** Order
- Company **Located_IN** City



Neo4j

Relationships can also have zero or more properties.

Two relationships of the same type can have different properties.

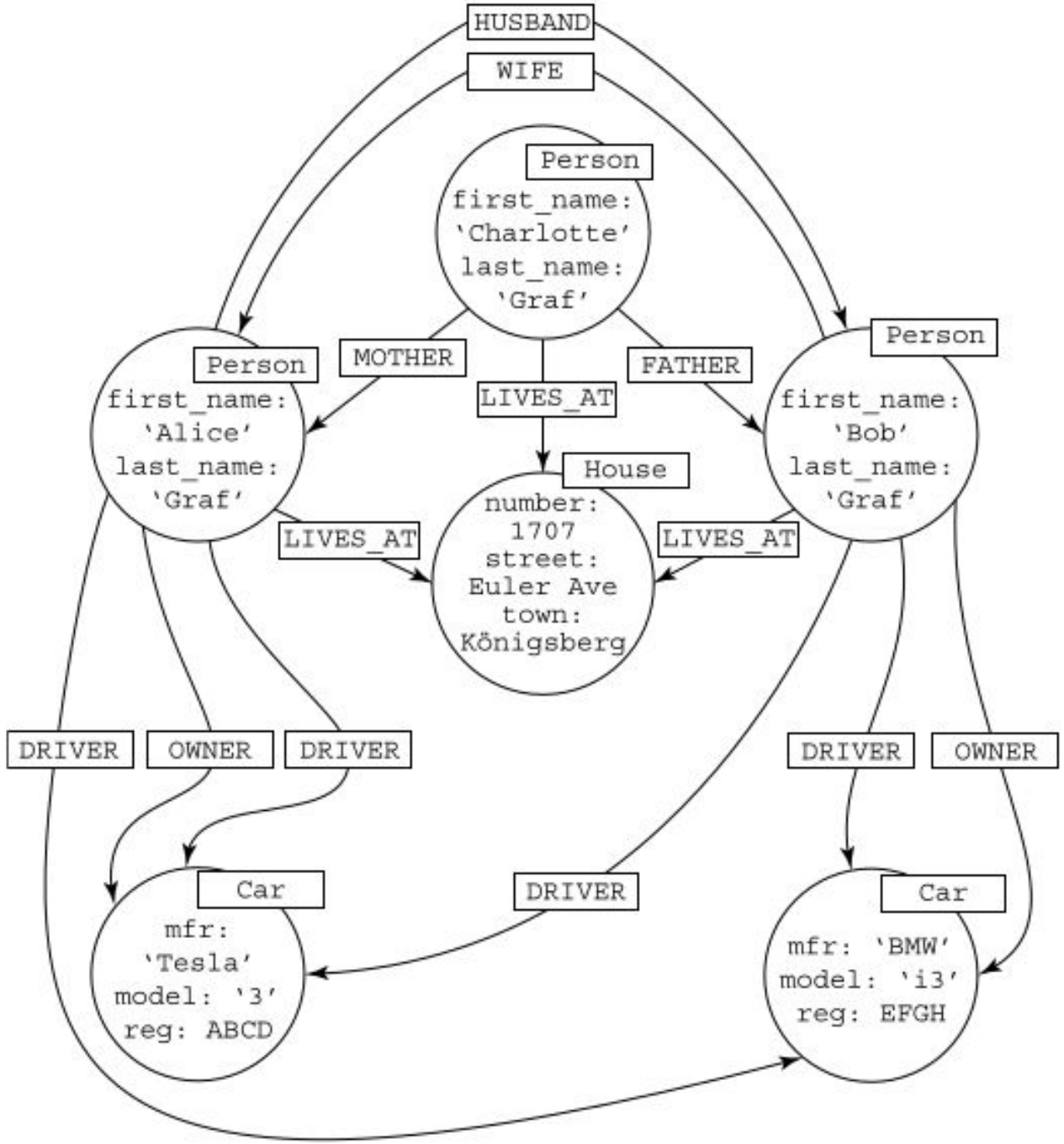


Neo4j

Borrowed from *Graph Databases For Dummies*

What types of nodes exist in this graph?

What types of relationships?



Neo4j - Retrieving Data

Retrieving data from a Neo4j database is done using the Cypher query language.

The basic idea is that you draw a picture of the pattern you want to find using ASCII art and then tell the database what to return.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

“MATCH” directs the database to look for a pattern.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

“MATCH” directs the database to look for a pattern.

A set of parentheses indicates a node. You can specify the type of node using :<Label>. Here, we are looking for a person.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

“MATCH” directs the database to look for a pattern.

A set of parentheses indicates a node. You can specify the type of node using :<Label>. Here, we are looking for a person.

We can also specify properties of the node we want to match by passing a set of key: Value pairs inside curly braces { }.

Here, we want the Person to be named “Alice”.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

Relationships are specified using square brackets [ ].

Similar to nodes, you can specify a label and/or properties of that relationship.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

Here, we are telling the database that we want to find any Person nodes that Alice is a friend of.

Notice that we gave this node a name (p). That way, we can refer to it later in the query.

Neo4j - Retrieving Data

```
MATCH (:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN p
```

Finally, we indicate that we want to return all of Alice’s friends.

To do this, we make use of the name that we gave the nodes in the MATCH statement.

Neo4j - Aggregating Data

Similar to SQL, we can aggregate the results of our queries.

```
MATCH (a:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN count(p)
```

```
MATCH (a:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
RETURN avg(p.age)
```

Neo4j - Aggregating Data

You can also filter the results using WHERE statements

```
MATCH (a:Person {name:'Alice'})-[:FRIEND_OF]->(p:Person)
WHERE p.age > 30
RETURN p
```

Neo4j

For more information about the Cypher language, see the Neo4j Cypher Developer Guide (or any of the other guides):

https://neo4j.com/developer/cypher/

A useful Cypher Cheat Sheet is available here:
https://neo4j.com/docs/cypher-cheat-sheet/5/neo4j-community

Some interesting cases are here:
https://neo4j.com/graphgists/

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

To see the whole database, you can use
MATCH(p) RETURN p;

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

1. Write a query to return the Keanu Reeves node.

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

1. Write a query to return the Keanu Reeves node.
2. Write a query to return all movies that Keanu Reeves acted in.

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

1. Write a query to return the Keanu Reeves node.
2. Write a query to return all movies that Keanu Reeves acted in.
3. Return all the people who directed movies that Keanu Reeves was in.

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

1. Write a query to return the Keanu Reeves node.
2. Write a query to return all movies that Keanu Reeves acted in.
3. Return all the people who directed movies that Keanu Reeves was in.
4. Find all actors that acted in at least two movies with Keanu Reeves.

Neo4j - Examples

Let’s walk through some examples of Cypher queries using the movies database.

1. Write a query to return the Keanu Reeves node.
2. Write a query to return all movies that Keanu Reeves acted in.
3. Return all the people who directed movies that Keanu Reeves was in.
4. Find all actors that acted in at least two movies with Keanu Reeves.
5. Find the shortest path between Keanu Reeves and Kevin Bacon.